

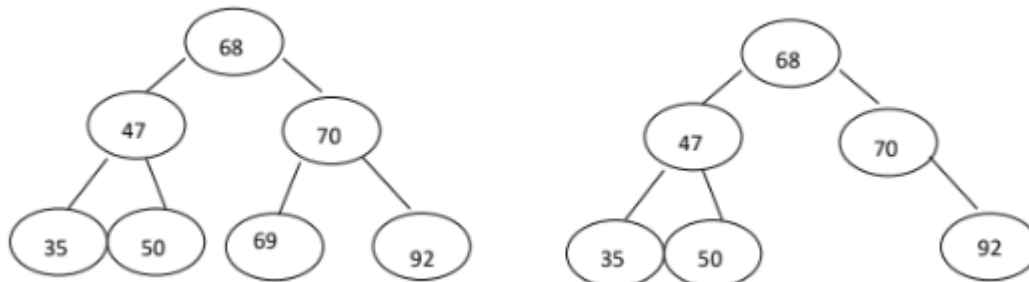
ÁRBOL

ESTRUCTURAS DE DATOS
y ALGORITMOS
LCC – LSI - TUPW

Árbol Balanceado - AVL

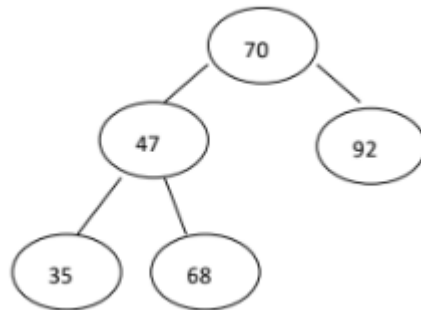
Los árboles balanceados reciben el nombre de **Árboles AVL** por ser Adelson-Velski y Landis quienes propusieron esta definición de equilibrio.

Un árbol es *perfectamente equilibrado*, si para cada nodo los números de nodos en sus subárboles izquierdo y derecho difieren cuanto más en uno.



Árbol Balanceado - AVL

Un árbol está *balanceado* si y solo si en cada nodo las alturas de sus dos subárboles difieren a lo máximo en uno.



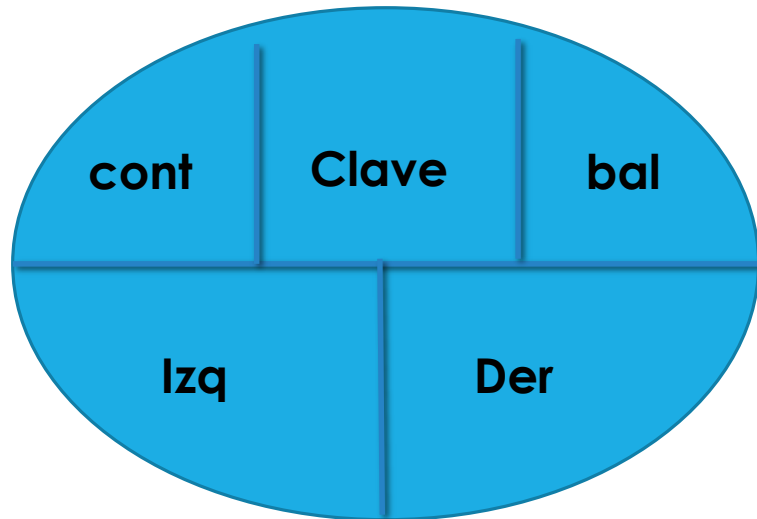
T.A.D. Árbol Balanceado

Definición del Objeto de Dato

El nodo incorpora un campo **Balanceo** que contiene información sobre la diferencia de las alturas de los subárboles del nodo

Nodo r

Campo Balanceo(r) = altura($D(r)$) – altura($I(r)$)



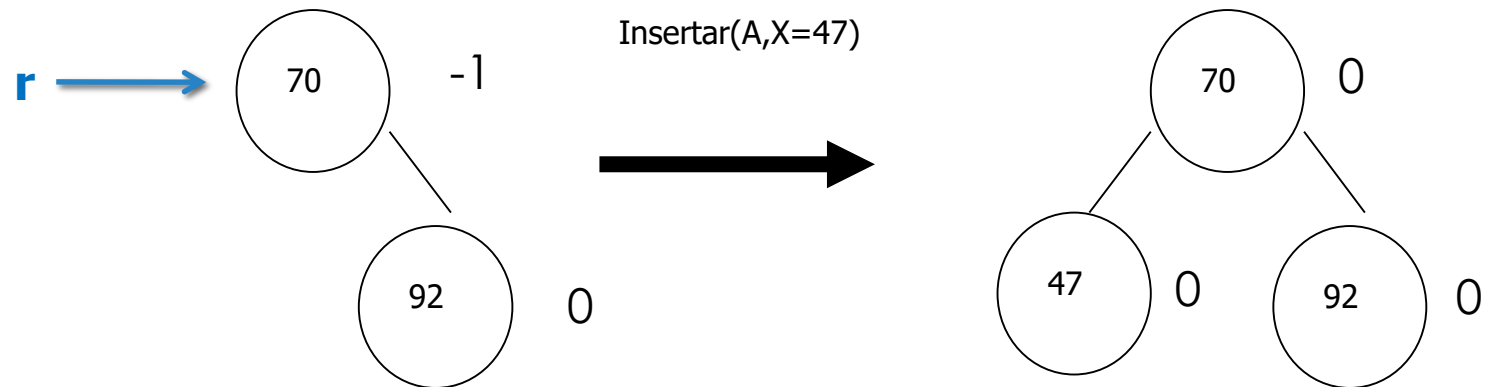
bal {
-1 si crece la rama derecha
0 si está balanceado
1 si crece la rama izquierda

T.A.D. Árbol Balanceado

Construcción de operaciones abstractas (1)

Insertión en un Árbol Balanceado

a) Si $\text{altura}(\text{I}(\text{r})) < \text{altura}(\text{D}(\text{r}))$ y X se inserta en $\text{I}(\text{r})$

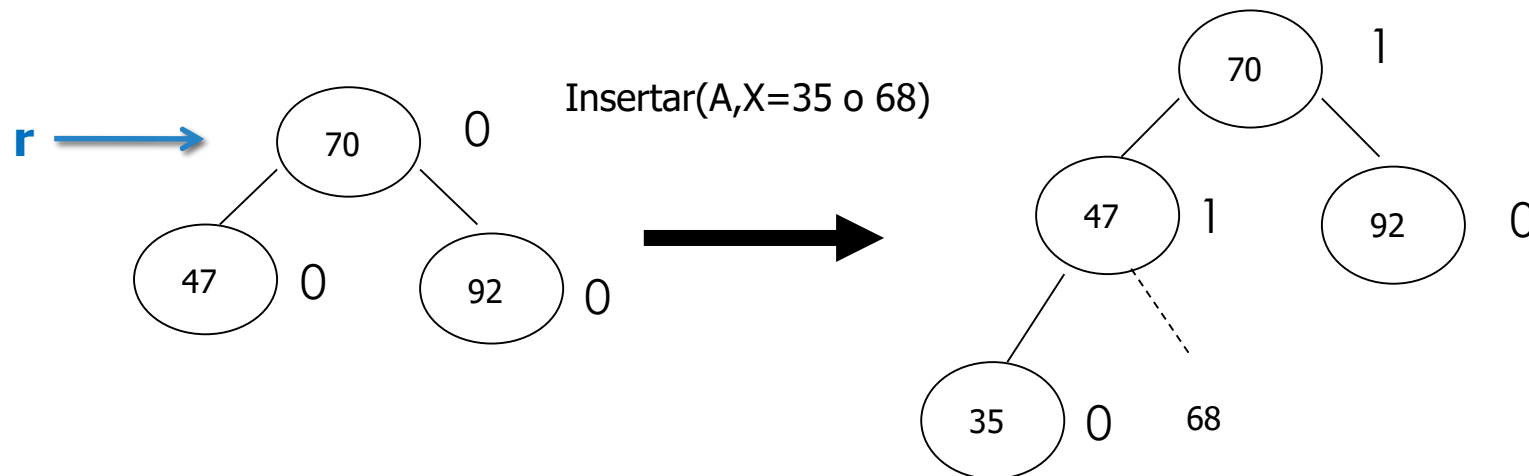


T.A.D. Árbol Balanceado

Construcción de operaciones abstractas (2)

Insertión en un Árbol Balanceado

b) Si $\text{altura}(\text{I}(\text{r})) = \text{altura}(\text{D}(\text{r}))$ y X se inserta en $\text{I}(\text{r})$,

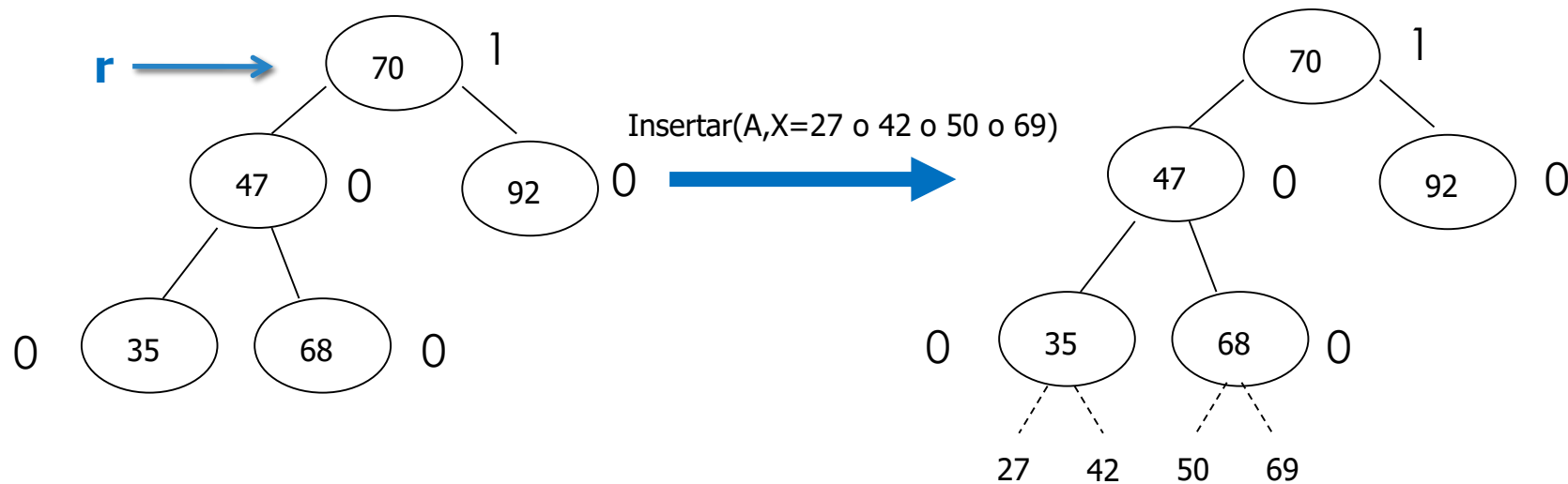


T.A.D. Árbol Balanceado

Construcción de operaciones abstractas (3)

Insertión en un Árbol Balanceado

c) Si $\text{altura}(I(r)) > \text{altura}(D(r))$ y X se inserta en $I(r)$



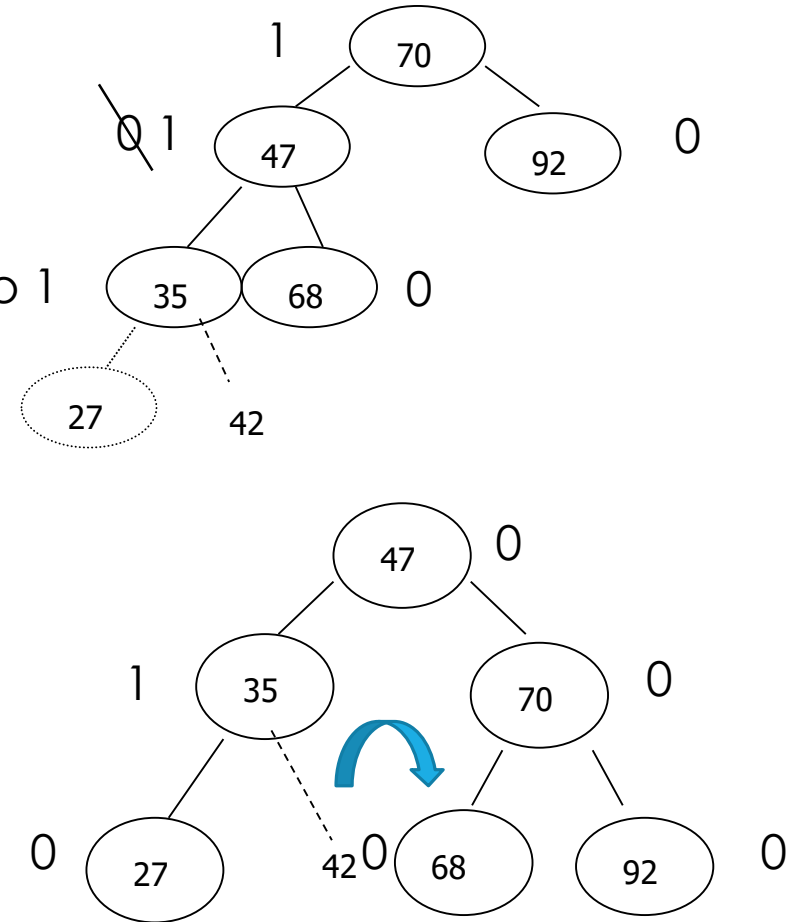
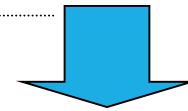
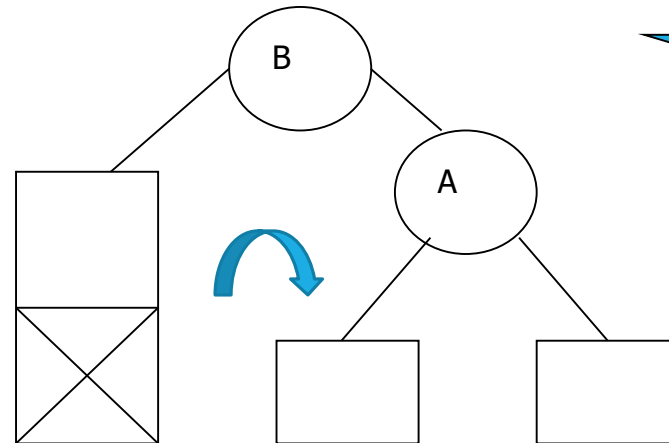
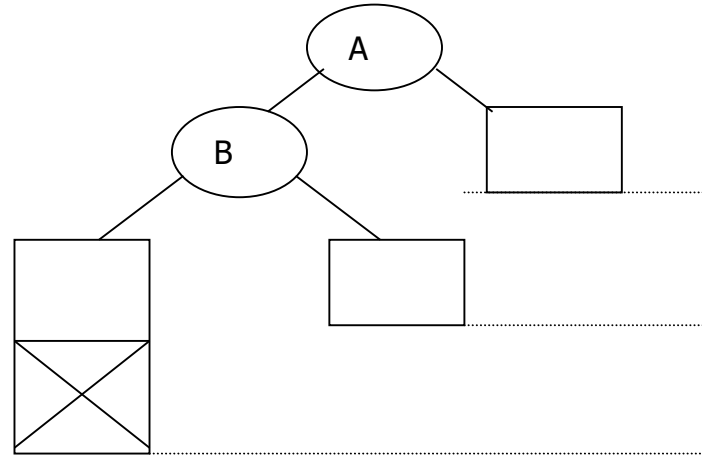
REBALANCEAR !!

T.A.D. Árbol Balanceado

Construcción de operaciones abstractas (4)

R
E
B
A
L
A
N
C
E
O

C
A
S
O
1

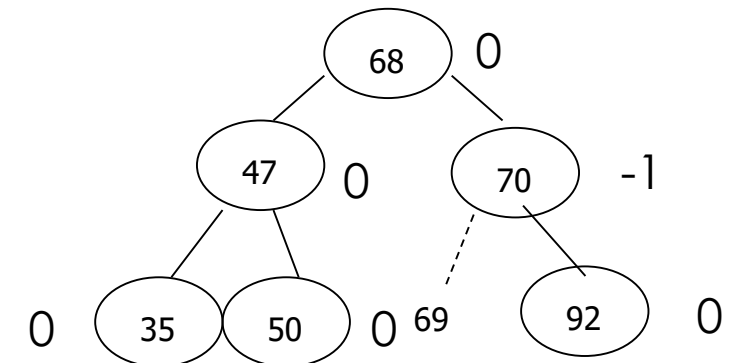
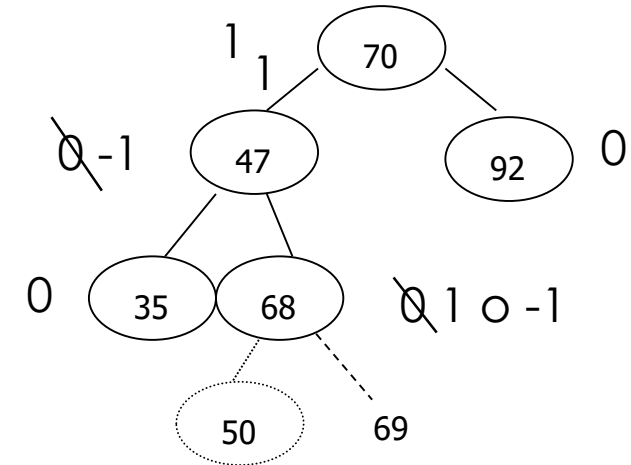
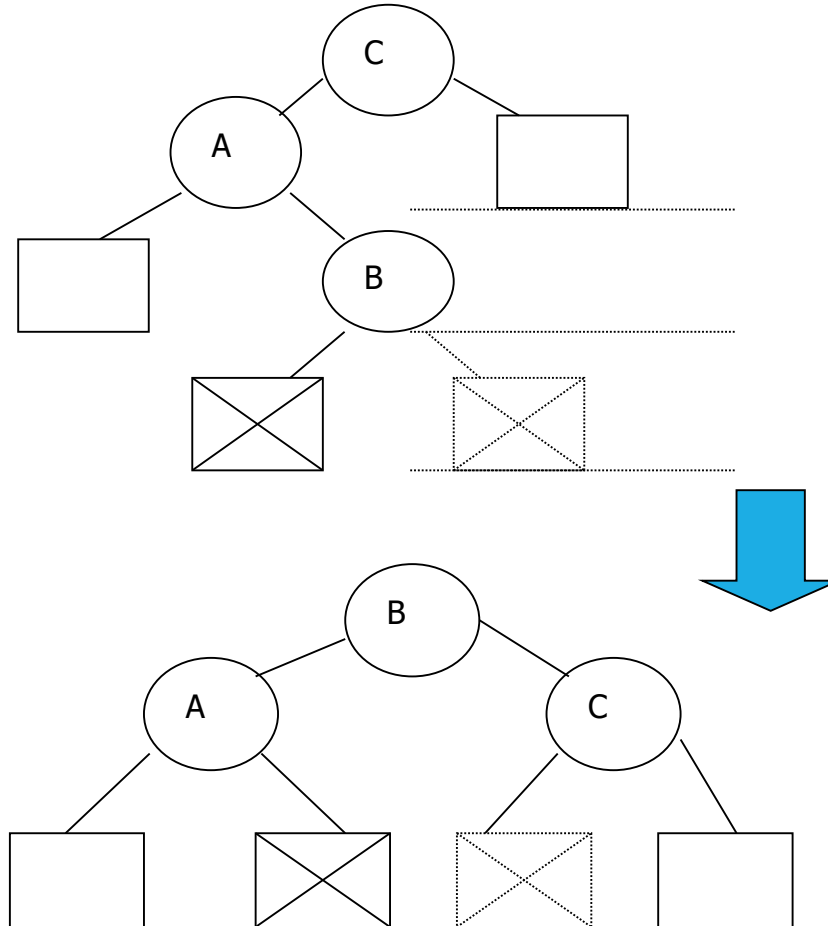


T.A.D. Árbol Balanceado

Construcción de operaciones abstractas (5)

REBALANCEO

CASO 2



Árbol Multicamino

Se utiliza cuando se necesita la Construcción y mantención de árboles de búsqueda a gran escala, que se almacenan en memoria secundaria donde:

Almacenar datos de 1 millón de elementos árbol balanceado

→ $\log_2 10^6 = 20$ comparaciones
acceso a disco por cada comp . 20 accesos a disco

Se incorpora un tipo particular de árbol multicamino

cantidad de accesos en el peor de los casos sería: $\log_{100} 10^6 = 3$ accesos

Cada página (excepto la raíz) contiene entre **n y $2n$** nodos para determinada constante **n** .

De ahí que, en un árbol con N elementos y un tamaño máximo de página de $2n$ nodos por página, en el peor caso requiere $\log_n N$ accesos de página.

Árbol B, árbol multiamino de orden n :

Se utilizan para gestionar almacenamiento en memoria secundaria, o búsqueda en bases de datos

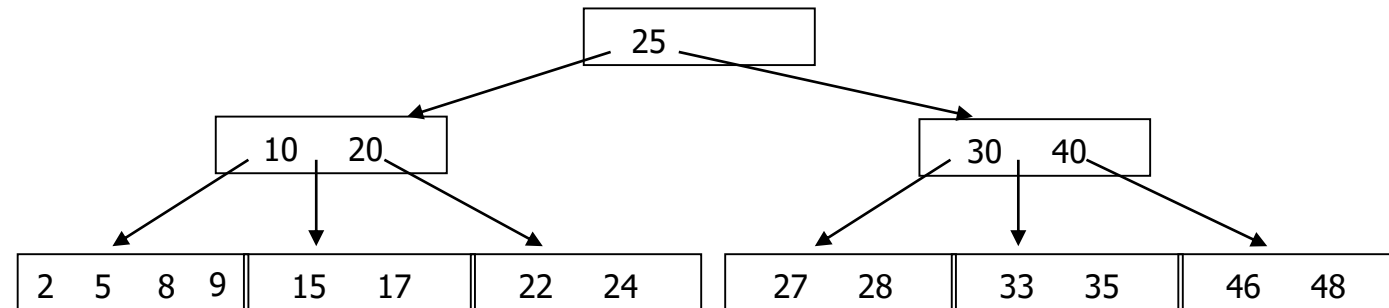
Su complejidad siempre será $O(\log n)$

- Cada página contiene a lo sumo $2n$ elementos (claves).
- Cada página, excepto la página raíz, contiene n elementos por lo menos.
- Cada página es una página de hoja, o sea que no tiene descendientes, o tiene $m+1$ descendientes, donde m es su número de claves en esa página ($n \leq m \leq 2n$).
- Todas las páginas hoja aparecen al mismo nivel.

T.A.D. Árbol B de orden $n=2$

Especificación

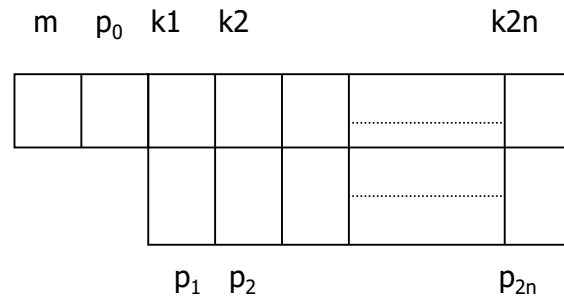
- Cada página contiene a lo sumo 4 ($2*2$) elementos (claves).
- Cada página, excepto la pagina raíz, contiene 2 elementos por lo menos.
- Cada página es una página de hoja, o sea que no tiene descendientes, o tiene $m+1$ descendientes, donde m es su número de claves en esa página ($2 \leq m \leq 4$).
- Todas las páginas hoja aparecen al mismo nivel.



T.A.D. Árbol B

Representación

Estructura de la Página



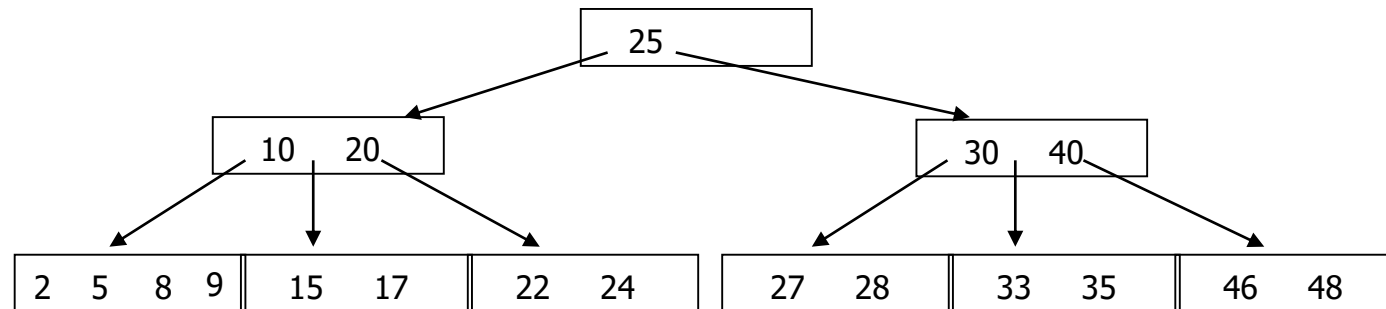
m : cantidad de claves en la página

k_i : clave; 1 ≤ i ≤ m

p₀ : dirección de la página que contiene claves menores que k₁

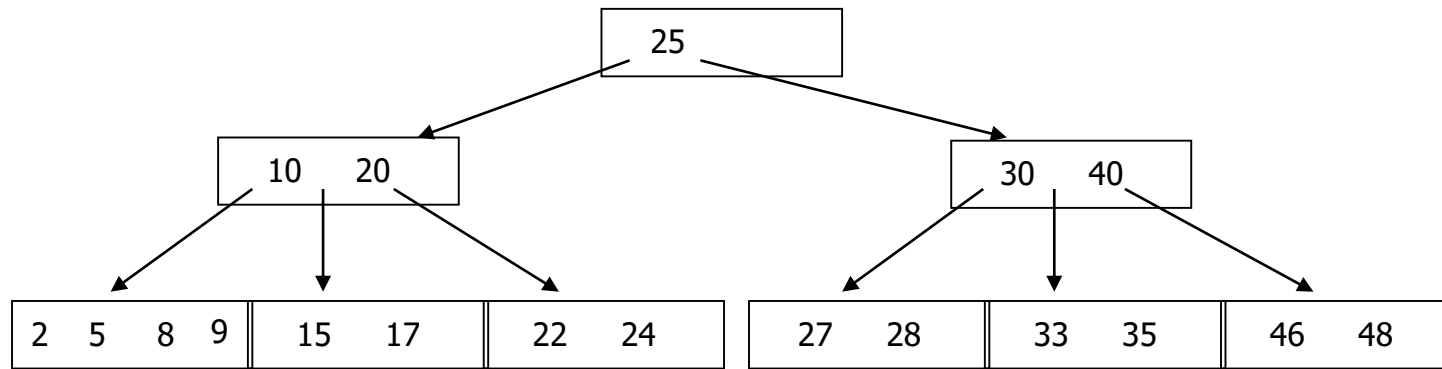
p_i : dirección de la página que contiene claves mayores que k_i y menores que k_{i+1}

p_m : dirección de la página que contiene claves mayores que k_m



T.A.D. Árbol B

Operaciones Abstractas: Buscar



¿Cómo se realiza la búsqueda de una clave X , en una página dada?

- 1) $k_m < X$;
- 2) $X < k_1$;
- 3) $k_i < X < k_{i+1}$ ($1 \leq i < m$)

¿Si la clave X no se encuentra en una página, cómo continúa la búsqueda?

entonces la búsqueda continúa por la página apuntada por p_m
entonces la búsqueda continúa por la página apuntada por p_0
entonces la búsqueda continúa por la página apuntada por p_i

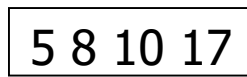
T.A.D. Árbol B

Operaciones Abstractas: Insertar

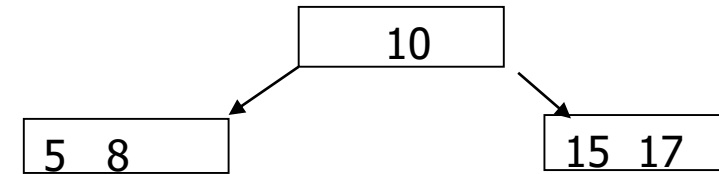
En este tipo de estructura los nodos se insertan en las hojas. El crecimiento es desde las hojas hacia la raíz

Insertamos los nodos: 8,10, 5, 17, 15, 24, 20, 9, 48, 22, 30, 2, 46, 28, 25, 27, 33, 40, 35

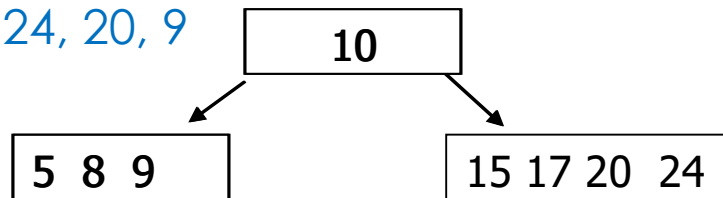
Inserta 8,10, 5, 17,



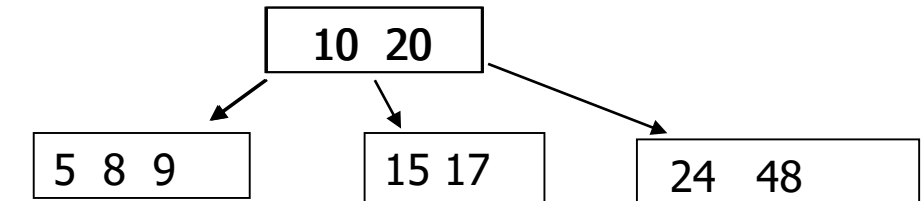
si insertamos el 15



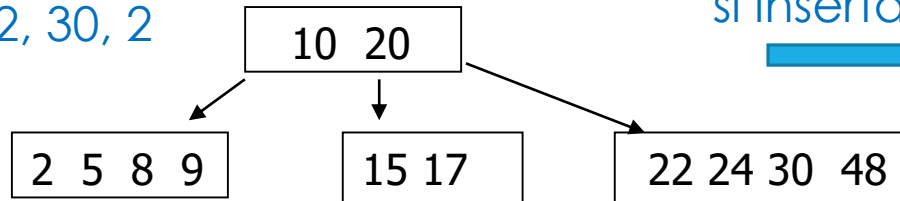
Inserta 24, 20, 9



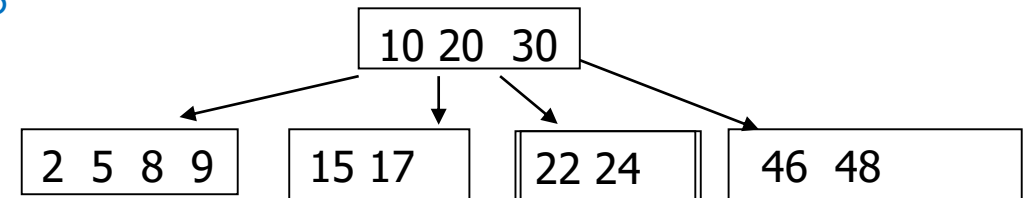
si insertamos el 48



Inserta 22, 30, 2



si insertamos el 46



T.A.D. Montículo Binario

- *Un Montículo Binario es un tipo de árbol binario, que ha diferencia de los vistos anteriormente, es almacenado secuencialmente, es decir, prescinde de enlaces.*
- *El montículo binario es un tipo de árbol binario para el cual se adopta la representación secuencial de objeto de datos, al construir el TAD.*
- Un Montículo Binario es ideal para implementar colas de prioridad, ya que garantiza que el elemento con la máxima o mínima prioridad (dependiendo de si es montículo máximo o mínimo) esté siempre en la raíz, listo para ser extraído.
- La estructura montículo binario garantiza que el tiempo mínimo se las operaciones insertar y eliminar sea de complejidad $O(\log n)$.

T.A.D. Montículo Binario

Por que es útil ?

- *Acceso rápido a la prioridad*

Permite encontrar y extraer el elemento con mayor o menor prioridad en tiempo constante $O(1)$, lo cual es super eficiente.

- *Implementación eficiente en memoria*

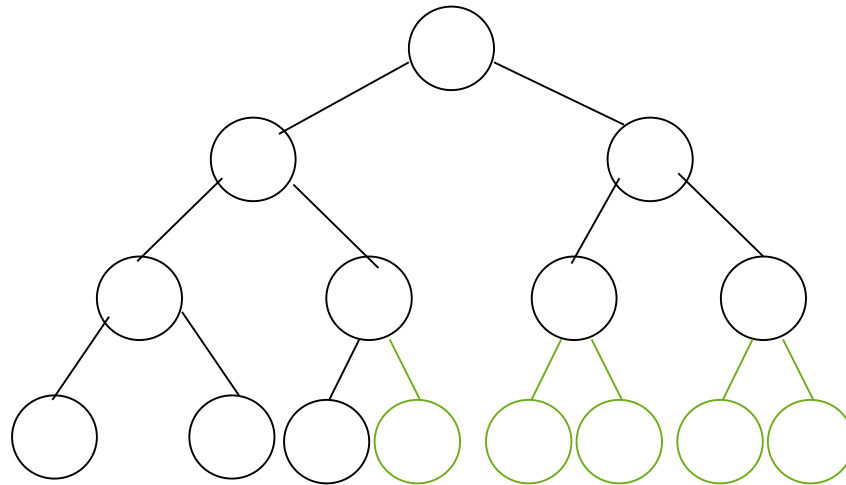
Los montículos binarios se pueden representar directamente en un arreglo (vector), sin necesidad de punteros, lo economiza memoria

- *Eficiencia en la adición y eliminación*

Añadir y suprimir elementos de un montículo binario tiene una complejidad de tiempo $O(\log n)$, ya que solo se necesita recorrer una fracción de los niveles del árbol

T.A.D. Montículo Binario

Árbol Binario Semicompleto: Un Árbol Binario Semicompleto de n nodos, se forma a partir de un árbol binario completo de $n+q$ nodos, quitando las q hojas extremo derechas del árbol binario completo.



$N=15$

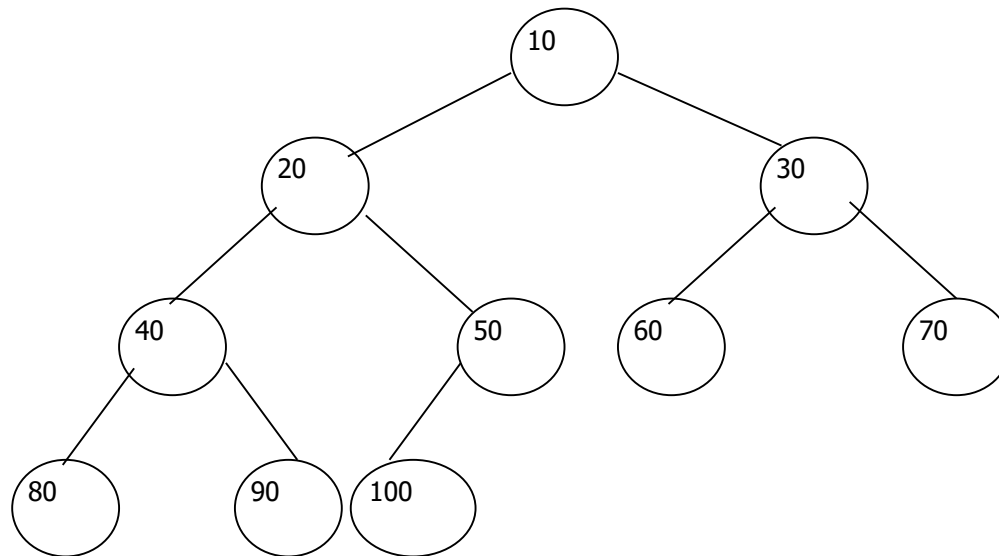
$n=10$

$q= 5$

T.A.D. Montículo Binario

Especificación

Montículo Binario: Un *Montículo Binario* es un árbol binario semicompleto en el que el valor de clave almacenado en cualquier nodo es menor o igual que el valor de clave de sus hijos.



Montículos Binarios Colas de Prioridad

Cola de procesos → Lista FIFO

Procesos **urgentes** → ~~Lista FIFO~~ → Cola de Prioridad

↓

cuantificado por medio de un valor
prioridad

Clave (montículo binario) → **prioridad**

Los valores –claves- que representan prioridades deben interpretarse de la siguiente manera: **a menor valor-mayor prioridad**, por lo que **la máxima prioridad se encuentra en la raíz del árbol**.

T.A.D. Montículo Binario Especificación

Operaciones Abstractas

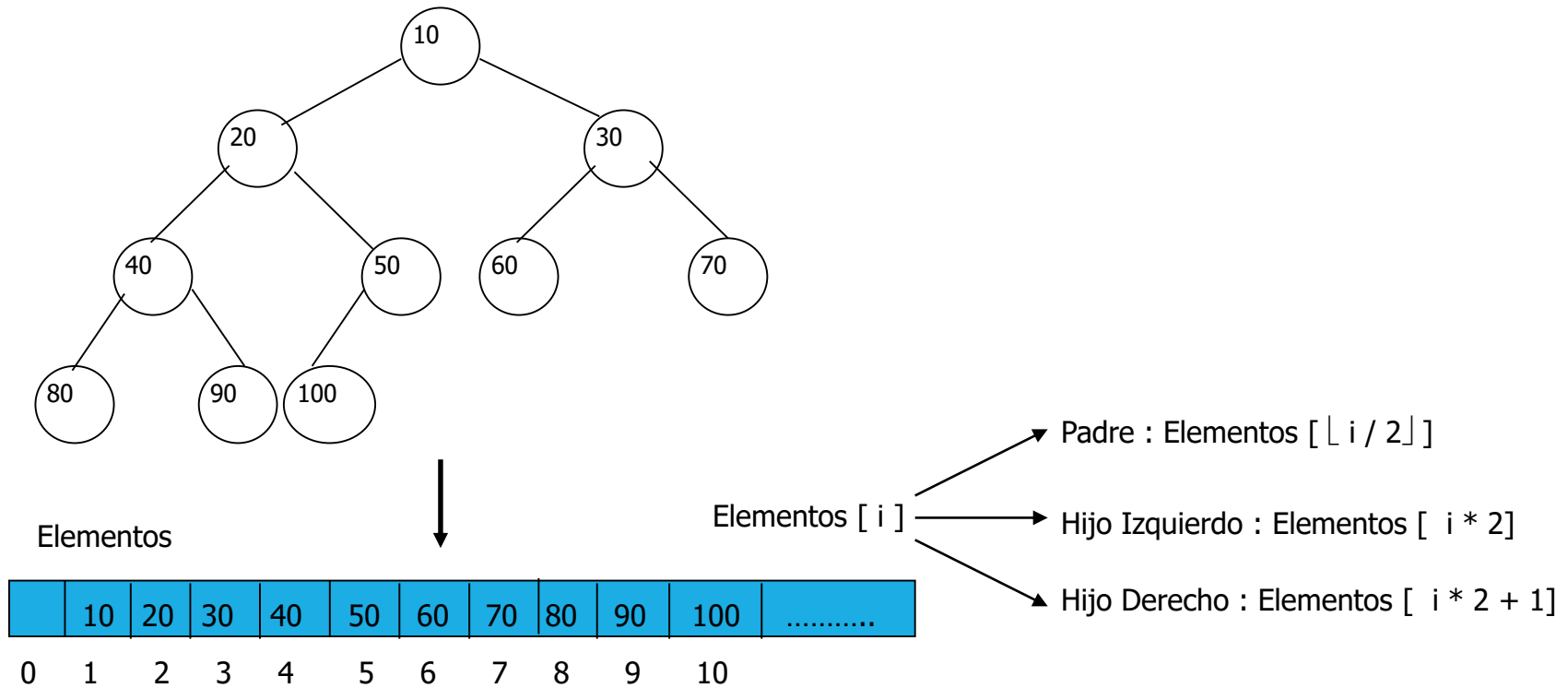
M: Montículo Binario y X: Clave

<i>NOMBRE</i>	<i>ENCABEZADO</i>	<i>FUNCIÓN</i>	<i>ENTRADA</i>	<i>SALIDA</i>
Insertar	Insertar(M, X)	Ingresa el elemento X al montículo M	M, X	M con el nuevo elemento
Eliminar_Mínimo	Eliminar_Mínimo(M, X)	Suprime del montículo M el elemento de máxima prioridad- mínimo valor de clave	M	M y X: elemento de máxima prioridad

T.A.D. Montículo Binario

Representación

Montículo Binario desde una perspectiva conceptual



Montículo Binario en su almacenamiento

T.A.D. Montículo Binario

Construcción de operaciones abstractas (1)

Tanto la operación ***Insertar*** como ***Eliminar_Mínimo***, deben garantizar que en el Montículo Binario se mantengan las siguientes dos propiedades:

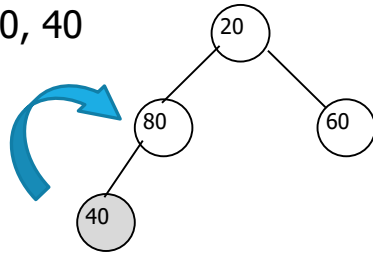
- ***Propiedad de Estructura*** : el objeto de datos debe ser un árbol binario semicompleto.
- ***Propiedad de Orden*** : el valor de clave almacenado en cualquier nodo debe ser menor o igual que el valor de clave de sus hijos.

T.A.D. Montículo Binario

Construcción de operaciones abstractas (2)

Insertar 20, 80, 60, 40, 50, 30, 70, 10, 90, 100, 35

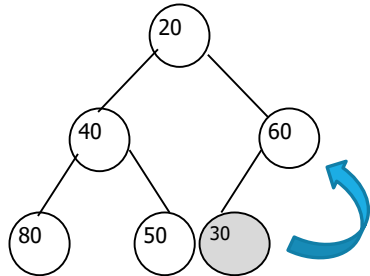
Inserto 20, 80, 60, 40



Elementos

	20	80	60	40								
0	1	2	3	4	5	6	7	8	9	10	11	

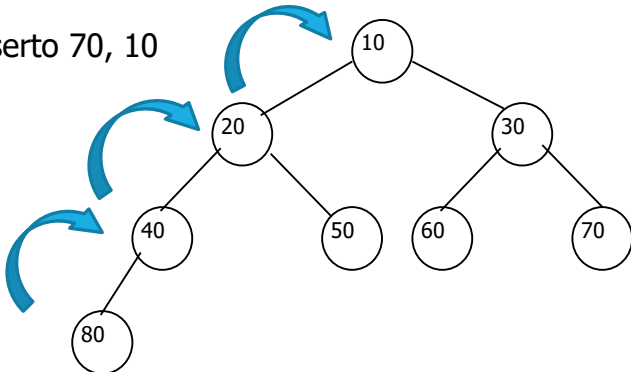
Inserto 50, 30



Elementos

	20	40	60	80	50	30						
0	1	2	3	4	5	6	7	8	9	10	11	

Inserto 70, 10



Elementos

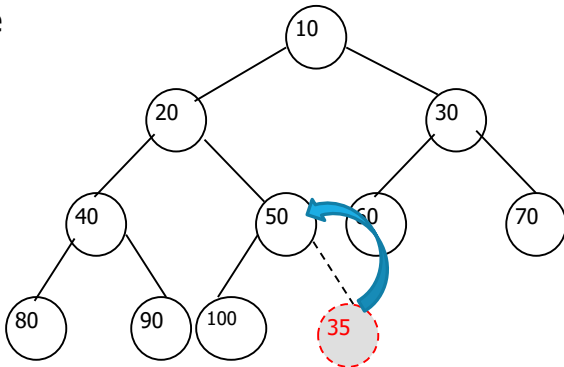
	20	40	30	80	50	60	70	10				
	20	40	30	10	50	60	70	80				
	20	40	30	10	50	60	70	80				
	10	20	30	40	50	60	70	80				
0	1	2	3	4	5	6	7	8	9	10	11	

T.A.D. Montículo Binario

Construcción de operaciones abstractas (3)

Insertar ($M, X=35$)

a) Propiedad de Estructura



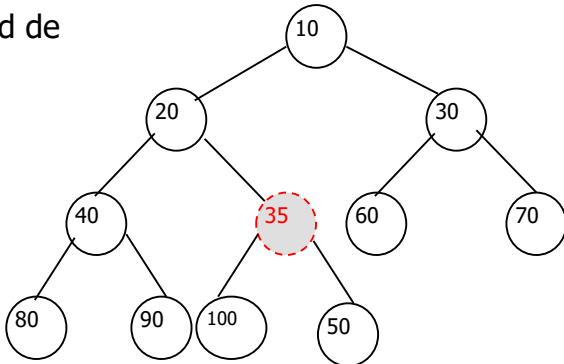
Elementos

	10	20	30	40	50	60	70	80	90	100	
0	1	2	3	4	5	6	7	8	9	10	11

Elementos

	10	20	30	40	50	60	70	80	90	100	35	
0	1	2	3	4	5	6	7	8	9	10	11	

b) Propiedad de Orden



Elementos

	10	20	30	40	35	60	70	80	90	100	50	
0	1	2	3	4	5	6	7	8	9	10	11	

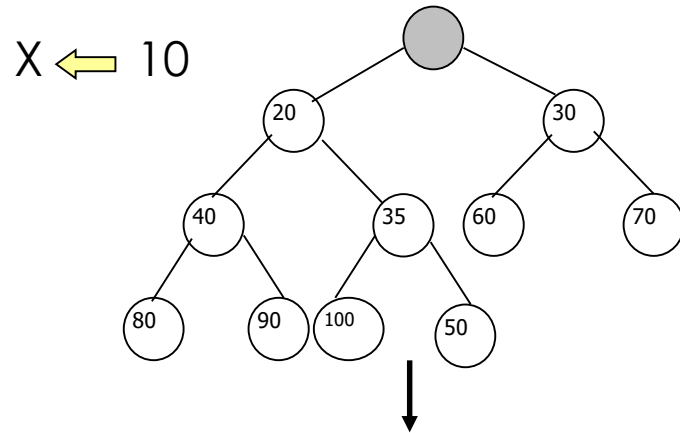
T.A.D. Montículo Binario

Construcción de operaciones abstractas (4)

Eliminar_Mínimo (M, X)

Elementos

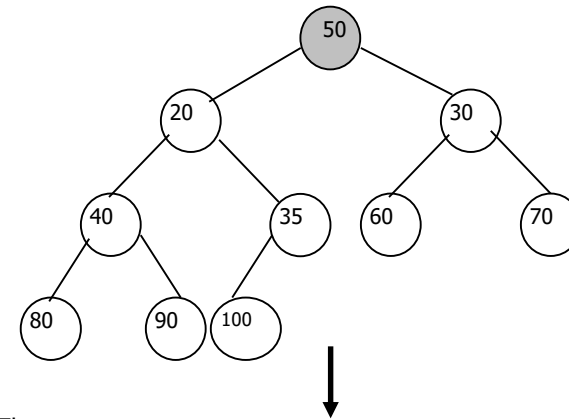
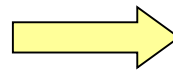
	10	20	30	40	35	60	70	80	90	100	50	
0	1	2	3	4	5	6	7	8	9	10	11	



Elementos

	20	30	40	35	60	70	80	90	100	50	
0	1	2	3	4	5	6	7	8	9	10	11

a) Propiedad de Estructura



Elementos

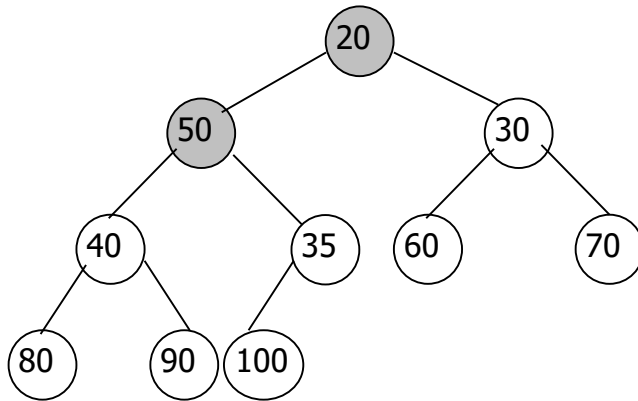
	50	20	30	40	35	60	70	80	90	100	
0	1	2	3	4	5	6	7	8	9	10	11

T.A.D. Montículo Binario

Construcción de operaciones abstractas (5)

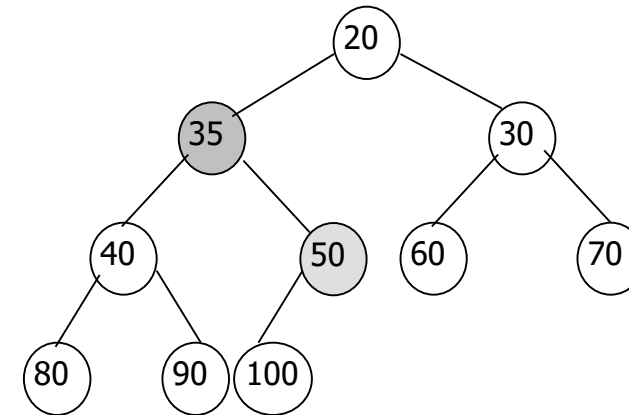
Eliminar_Mínimo (M, X)

b) Propiedad de Orden



Elementos

	20	50	30	40	35	60	70	80	90	100		
0	1	2	3	4	5	6	7	8	9	10	11	



Elementos

	20	35	30	40	50	60	70	80	90	100		
0	1	2	3	4	5	6	7	8	9	10	11	

T.A.D. Montículo Binario

Ejemplos de aplicación

- En los hospitales, para administrar la sala de emergencia: los pacientes se atienden según gravedad de su estado (mayor prioridad primero)
- En el sistema operativo, los procesos en la CPU se pueden manejar con prioridades para decidir cual ejecutar antes
- En GPS y mapas (Google Maps, Waze, etc)
- Las colas de impresión, donde algunos documentos tienen mas prioridad que otros
- En simuladores (por ejemplo, de tráfico aéreo), se ordenan los eventos futuros según el tiempo en que ocurrirán, usando un montículo para siempre acceder al próximo evento mas cercano
- En bolsas de valores, para manejar ordenes de compra/venta con distintas prioridades